- **Cause**: Overwriting or saving errors when exporting CSV/video.
- **Solution**: Using external tools like OBS Studio or Windows Snippet tool for screenshot.

6. Future Design Considerations

While the design supports the minimum viable product (MVP) for human-centered motion annotation, additional design features could be considered:

Cloud Integration: Upload annotations directly to cloud storage or research databases.

Multi-Camera Support: Add support for external webcams or multiple angles.

Gesture Recognition: Leverage MediaPipe or PoseNet for automatic annotations.

Export Formats: Allow export to JSON or XML for integration with ML pipelines.

7. Summary

This appendix provided a comprehensive overview of the design rationale, functional capabilities, system architecture, and technical decisions behind the video annotation tool. The clarity in system flow and modularity in implementation allowed the tool to fulfill its purpose in a research-oriented context, particularly for applications in human-centred computing, gesture annotation, and video-based behaviour analysis. The system meets the initial design goals and lays a strong foundation for future development and scaling.

Appendix B

Code and Implementation Breakdown

This appendix documents the implementation of the video motion capture and annotation tool. Each section corresponds to the code cells from the Jupyter Notebook and provides an explanation of their functionality and rationale.

A1. Installation of Dependencies

```
[7]: !pip install mediapipe opencv-python

Requirement already satisfied: mediapipe in c:\users\elvis\anaconda3\lib\site-packages (0.10.21)
Requirement already satisfied: opencv-python in c:\users\elvis\anaconda3\lib\site-packages (4.11.0.86)
Requirement already satisfied: absl-py in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (2.2.2)
Requirement already satisfied: attrs>=19.1.0 in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (23.1.0)
Requirement already satisfied: flatbuffers>=2.0 in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (25.2.10)
Requirement already satisfied: jax in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (0.6.0)
Requirement already satisfied: jaxlib in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (0.6.0)
Requirement already satisfied: matplotlib in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (3.9.2)
Requirement already satisfied: numpy<2 in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (1.26.4)
Requirement already satisfied: opencv-contrib-python in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (4.11.0.86)
Requirement already satisfied: protobuf<5,>=4.25.3 in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (4.25.3)
Requirement already satisfied: sounddevice>=0.4.4 in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (0.5.1)
Requirement already satisfied: sentencepiece in c:\users\elvis\anaconda3\lib\site-packages (from mediapipe) (0.2.0)
Requirement already satisfied: CFFI>=1.0 in c:\users\elvis\anaconda3\lib\site-packages (from sounddevice>=0.4.4->mediapipe) (1.17.1)
Requirement already satisfied: ml_dtypes>=0.5.0 in c:\users\elvis\anaconda3\lib\site-packages (from jax->mediapipe) (0.5.1)
Requirement already satisfied: opt_einsum in c:\users\elvis\anaconda3\lib\site-packages (from jax->mediapipe) (3.4.0)
Requirement already satisfied: scipy>=1.11.1 in c:\users\elvis\anaconda3\lib\site-packages (from jax->mediapipe) (1.13.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\elvis\anaconda3\lib\site-packages (from matplotlib->mediapipe) (2.9.0.post0)
Requirement already satisfied: pycparser in c:\users\elvis\anaconda3\lib\site-packages (from CFFI>=1.0->sounddevice>=0.4.4->mediapipe) (2.21)
Requirement already satisfied: six>=1.5 in c:\users\elvis\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->mediapipe) (1.16.0)
```

This command installs the two main Python libraries required for the project:

MediaPipe: A Google framework used for real-time perception pipelines such as face detection, pose estimation, and hand tracking.

OpenCV: A widely used computer vision library that enables image and video processing.

These packages provide the tools for capturing webcam feeds and analyzing body landmarks.

A2. Importing Required Libraries and Initializing MediaPipe Components

```
[2]: import mediapipe as mp
     import cv2
     import numpy as np

[3]: mp_drawing = mp.solutions.drawing_utils
     mp_holistic = mp.solutions.holistic
     mp_drawing_styles = mp.solutions.drawing_styles
```

This cell imports the necessary modules:

mediapipe as mp: Used for accessing holistic models, drawing utilities, and landmark styles.

cv2: The OpenCV library, which handles video capture, drawing on frames, and displaying windows.

numpy: Used for numerical operations, though not heavily used in this implementation.

drawing_utils: Utilities to render landmark points and connections on frames.

holistic: The holistic model that combines face, hands, and pose detection into a single pipeline.

drawing_styles: Contains pre-defined drawing styles for better visual clarity.

A4. Raw Webcam Feed Preview

```
[6]: cap = cv2.VideoCapture(0)
     while cap.isOpened():
         ret, frame = cap.read()
         cv2.imshow('Raw Webcam Feed', frame)

         if cv2.waitKey(10) & 0xFF == ord('q'):
             break

     cap.release()
     cv2.destroyAllWindows()
```

This block sets up a live webcam stream:

It uses **cv2.VideoCapture(0)** to access the default webcam.

A loop reads and displays each frame in real-time.

Pressing the **'q'** key exits the loop and releases the camera.

This code ensures that the webcam is functional and integrated into the notebook workflow.

A5. Annotating Hands, Face, and Pose

```
# Define hand annotation function outside the loop
def draw_hand_box(image, hand_landmarks, label):
    if hand_landmarks:
        h, w, _ = image.shape
        coords = [(int(lm.x * w), int(lm.y * h)) for lm in hand_landmarks.landmark]
        x_coords, y_coords = zip(*coords)
        x_min, x_max = min(x_coords), max(x_coords)
        y_min, y_max = min(y_coords), max(y_coords)
        cv2.rectangle(image, (x_min-10, y_min-10), (x_max+10, y_max+10), (0, 255, 0), 2)
        cv2.putText(image, label, (x_min-10, y_min-20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

Hand Landmark Detection:

Input: The *draw_hand_box()* function takes two main arguments: image (the current frame) and *hand_landmarks* (which contains the detected landmarks for the hand).

Check for Presence: The function first checks whether the *hand_landmarks* object is None or contains valid landmarks. If it does, it proceeds with annotation.

Coordinate Conversion:

Normalized to Pixel: The hand landmarks provided by MediaPipe are normalized, with coordinates ranging from 0 to 1. To render them on the frame, these normalized values are converted to pixel coordinates using the width *(w)* and height *(h)* of the image (frame).

```
coords = [(int(lm.x * w), int(lm.y * h)) for lm in hand_landmarks.landmark]
```

This list comprehension extracts the x and y values from the landmarks and scales them by the width and height of the frame.

Bounding Box Calculation:

Coordinate Extraction: The coordinates of the landmarks are unpacked into *x_coords* and *y_coords* using the *zip()* function.

```
x_coords, y_coords = zip(*coords)
```

Bounding Box: To draw a bounding box around the hand, the minimum and maximum values of the x and y coordinates are calculated:

```
x_min, x_max = min(x_coords), max(x_coords)
y_min, y_max = min(y_coords), max(y_coords)
```

This gives the top-left *((x_min, y_min))* and bottom-right *((x_max, y_max))* coordinates for the bounding box.

Drawing the Bounding Box:

Rectangle: A rectangle is drawn using *cv2.rectangle()* to highlight the detected hand. The rectangle extends 10 pixels beyond the calculated bounds to give a better visual cue around the hand.

```
cv2.rectangle(image, (x_min-10, y_min-10), (x_max+10, y_max+10), (0, 255, 0), 2)
```

Labeling: The label (e.g., "Left Hand" or "Right Hand") is added slightly above the bounding box using *cv2.putText():*

```
cv2.putText(image, label, (x_min-10, y_min-20),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

The position of the text is set just above the bounding box (offset by *-10* pixels on the x and *-20* on the y), and the color *(0, 255, 0)* (green) is used for both the rectangle and the label.

This function allows the real-time display of detected hand landmarks with a visual bounding box and a label. It's useful for providing clear feedback to the user regarding the detection and tracking of hand landmarks, especially in a motion capture or human-computer interaction system.

A6. Initialize MediaPipe Holistic Model

```
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:
```

This context manager loads the **Holistic** model with:

***min_detection_confidence=0.5*:** Minimum confidence to detect new landmarks.

***min_tracking_confidence=0.5*:** Minimum confidence to keep tracking landmarks once detected.

Holistic combines face mesh, hand tracking, and pose estimation into a single pipeline.

A7. Read Frame Loop

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
```

The loop runs as long as the webcam stream is open.

cap.read() reads a single frame:

ret: Boolean flag indicating if frame capture was successful.

frame: The actual image array from the webcam.

If no frame is returned, the loop breaks

A8. Convert Frame Color Space for MediaPipe

```
# Convert BGR to RGB
image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = holistic.process(image)
```

OpenCV captures images in **BGR**, but MediaPipe expects **RGB**.

This conversion ensures compatibility with the holistic model.

holistic.process(image) applies the model to the frame and stores the results (landmarks) in *results*.

A9. Convert Back to BGR for Display

```
# Convert RGB back to BGR
image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
```

After landmark processing, the image is converted back to **BGR** for OpenCV to display correctly.

A10. Draw Bounding Boxes Around Hands

```
# Draw hand boxes
draw_hand_box(image, results.left_hand_landmarks, "Left Hand")
draw_hand_box(image, results.right_hand_landmarks, "Right Hand")
```

This uses the helper function draw_hand_box() (defined in A.5) to:

- Draw bounding boxes around left and right hands.
- Label them for clarity.

A11. Create Drawing Style for Facial Landmarks

```
# DrawingSpec with small dots
small_dots_spec = mp_drawing.DrawingSpec(thickness=1, circle_radius=1, color=(0, 255, 0))
```

A custom *DrawingSpec* defines how landmarks are drawn.

Here, the face mesh will be drawn with small green dots, making it less visually dense.

A12. Draw Landmarks on the Frame

Face Mesh

```
# Draw landmarks
mp_drawing.draw_landmarks(
    image, results.face_landmarks, mp_holistic.FACEMESH_CONTOURS,
    landmark_drawing_spec=small_dots_spec,
    connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_contours_style()
)
```

Draws the **face mesh contours** using MediaPipe's predefined style.

Custom *small_dots_spec* is applied to individual landmark points.

Hands

```
mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS)
mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS)
```

These lines draw both left and right hands, showing joint connections based on anatomical structure.

Pose Skeleton

```
mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS)
```

Draws the full pose skeleton, connecting key points like shoulders, hips, knees, and elbows.

A13.Display the Annotated Frame and Exit on Key Press

```
        #Once the letter q is pressed on the keyboard the program will terminate
        if cv2.waitKey(10) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()
```

Displays the annotated frame in a real-time preview window titled "Webcam Feed with Annotations".

Waits 10 milliseconds for a key press.

When the user presses the letter **q**, the loop exits.

cap.release(): Frees the webcam resource.

cv2.destroyAllWindows(): Closes any OpenCV windows that were opened during execution.

This is detailed breakdown of a helper function for hand annotation and implementation of real-time webcam capture with holistic landmark detection and live visual annotation.

Troubleshooting Note:

A recurring issue encountered during development was related to kernel inactivity or improper kernel resets, which could trigger webcam errors. If such errors occur, such as the webcam not initializing or the feed window not appearing, try the following steps:

Close and reopen the Jupyter Notebook.

Re-run the dependency installation cell **(pip install)** and the import cell by selecting them and pressing Alt+Enter.

If the laptop camera appears to be "in use" but no webcam window opens, it's likely the camera wasn't properly released. In this case, fully close Jupyter Notebook and reopen it.

If the issue persists, restart your machine to forcefully reset the integrated camera connection.